

Timothy Song

Professor Shim

DATA 260

21 September 2026

Homework 3 Report

GitHub Repository: <https://github.com/TimothyYSong/data260-9981>

Section 0 - Personal Configuration and Domain

Hardware: MacBook Pro M4 Pro, 14 cores (10 performance and 4 efficiency), 24 GB memory

Local model: qwen3:8b, Embedding model: sentence-transformers/all-MiniLM-L6-v2

Tagged commit hash: 7edc046327b8b0d14ba24897528633d7ff08a973

SID4: 9981

PORT_BASE: 8081

PREFIX: s9981

SEED: 9981

VERIFY_SEED: 269981

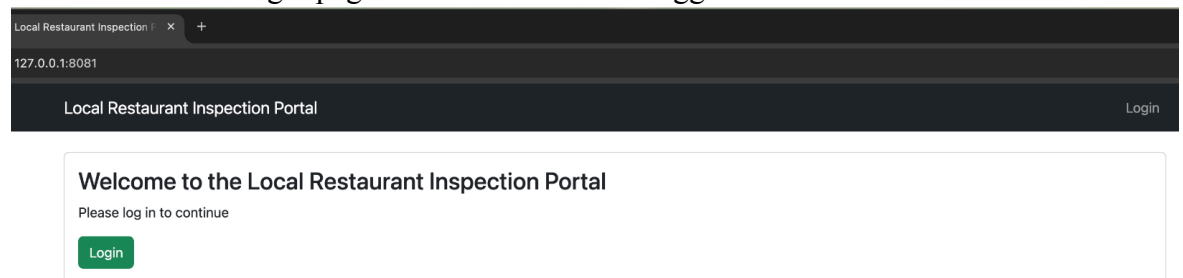
DOMAIN_ID: 5 (Local restaurant inspections)

Requirements:

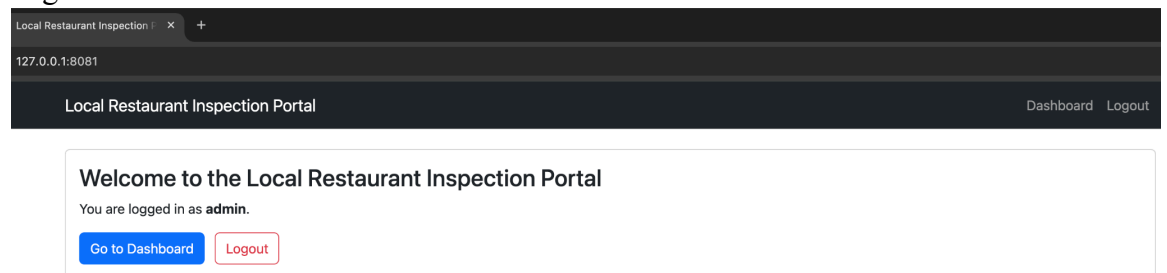
1. Routes - Handled Separately in a router:

- **Home Page (/):**

I display a welcome message for my Local Restaurant Inspections domain and show a link to the login page when the user is not logged in.

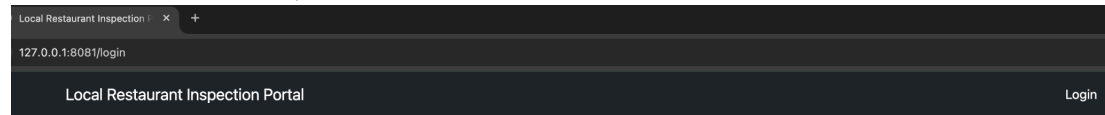


When the user is logged in, the Home page shows links to the Dashboard and Logout.



- **Login Page (/login):**

The login page displays fields for a username and password. When valid credentials are entered, the user is authenticated and redirected to the Dashboard.



Login
Username

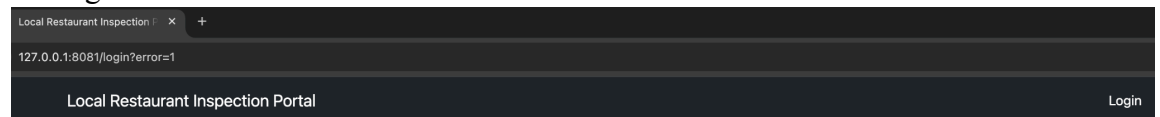
Password

[Back to Home](#)



Dashboard
Welcome admin

When invalid credentials are entered, the login page displays a Bootstrap alert message.



Login

Invalid username or password.

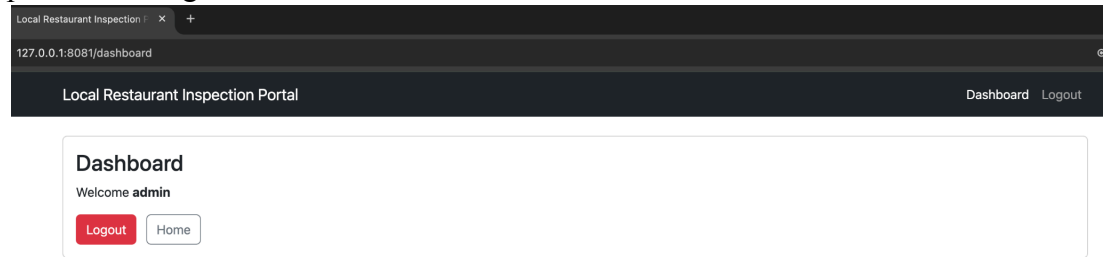
Username

Password

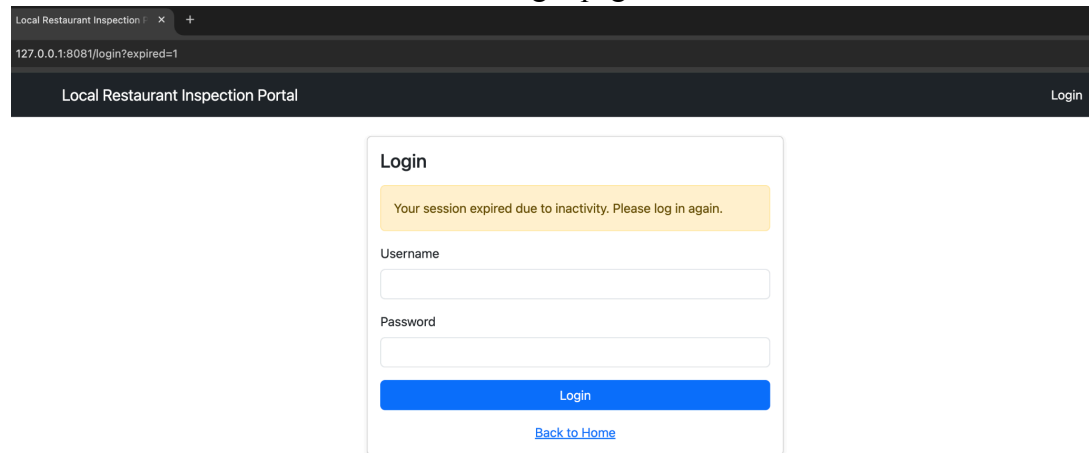
[Back to Home](#)

- **Dashboard Page (/dashboard):**

The Dashboard displays a welcome message with the logged-in user's name and provides a Logout link.

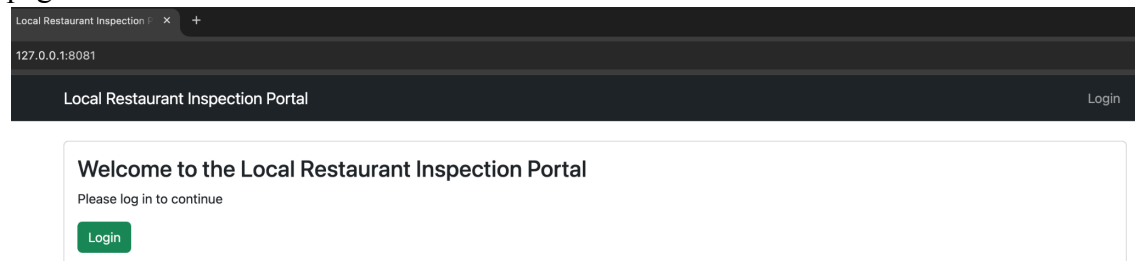


The /dashboard route is protected. When the session expires, attempting to access the Dashboard redirects the user to the login page.



- **Logout (/logout):**

After logging out, the session is destroyed and the user is redirected to the home page.

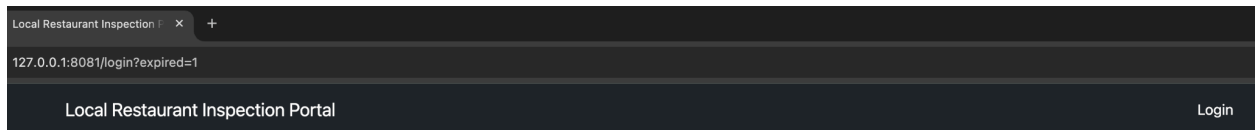


2. Session Management

I used Starlette's SessionMiddleware to manage sessions, and the logged-in username is stored in request.session. I configured the session cookie with `https_only=True`, which sets the cookie with the Secure attribute. The login response also shows `HttpOnly` and `SameSite=Lax` in the Set-Cookie header.

```
Last login: Fri Sep 18 18:56:57 on console
(base) optim@Timothys-MacBook-Pro ~ % curl -i -X POST \
  -d "username=admin&password=password" \
  [ http://127.0.0.1:8081/login ]
HTTP/1.1 302 Found
date: Mon, 21 Sep 2026 05:47:15 GMT
server: uvicorn
content-length: 0
location: /dashboard
vary: Cookie
set-cookie: session=eyJ1c2VyIjogImFkbWluIn0=.arDE4w.wOU9C90dieMBU_4GfBbb7jJC2MM;
  path=/; Max-Age=3600; httponly; samesite=lax; secure
```

After the idle timeout expires or after logging out, attempting to access `/dashboard` redirects the user to the login page. Both cases prevent access to the protected Dashboard route.



Login

Your session expired due to inactivity. Please log in again.

Username

Password

Login

[Back to Home](#)

3. Styling with Bootstrap:

All pages use Bootstrap components including navbars, cards, buttons, forms, and alerts, as shown in the route screenshots above.

Templates Directory and auth.py Code

```

auth.py U X
code > routers > auth.py > ...

1  import time
2  from pathlib import Path
3
4  from fastapi import APIRouter, Request, Form
5  from fastapi.responses import RedirectResponse
6  from fastapi.templating import Jinja2Templates
7  from starlette.status import HTTP_302_FOUND
8
9
10 # Create a router object
11 # This behaves like a mini FastAPI app
12 router = APIRouter()
13
14 # Configure Jinja2 templates directory
15 BASE_DIR = Path(__file__).resolve().parent.parent
16 templates = Jinja2Templates(directory=str(BASE_DIR / "templates"))
17
18
19 # Hardcoded credentials for demo purposes only
20 # In real applications, credentials come from a database
21 VALID_USERNAME = "admin"
22 VALID_PASSWORD = "password"
23
24 IDLE_TIMEOUT_SECONDS = 30
25
26
27 def get_active_user(request: Request):
28     user = request.session.get("user")
29     last_activity = request.session.get("last_activity")
30
31     if not user or last_activity is None:
32         return None
33
34     current_time = time.time()
35
36     if current_time - last_activity > IDLE_TIMEOUT_SECONDS:
37         request.session.clear()
38         return None
39
40     request.session["last_activity"] = current_time
41     return user
42
43
44 @router.get("/")
45 def home(request: Request):
46     """
47     Home page route.
48
49     - Checks if a user is logged in using the session
50     - Passes user info to the template
51     """
52     user = request.session.get("user")
53
54     return templates.TemplateResponse(
55         request,
56         "index.html",
57         {
58             "user": user
59         }
60     )
61
62
63 @router.get("/login")
64 def login_page(
65     request: Request,
66     error: str | None = None,
67     expired: str | None = None
68 ):
69     user = request.session.get("user")
70
71     return templates.TemplateResponse(
72         request,
73         "login.html",
74         {
75             "user": user,
76             "error": error,
77             "expired": expired
78         }
79     )

```

```

✓ templates
<> dashboard.html
<> index.html
<> login.html

```

```

auth.py U X
code > routers > auth.py > ...

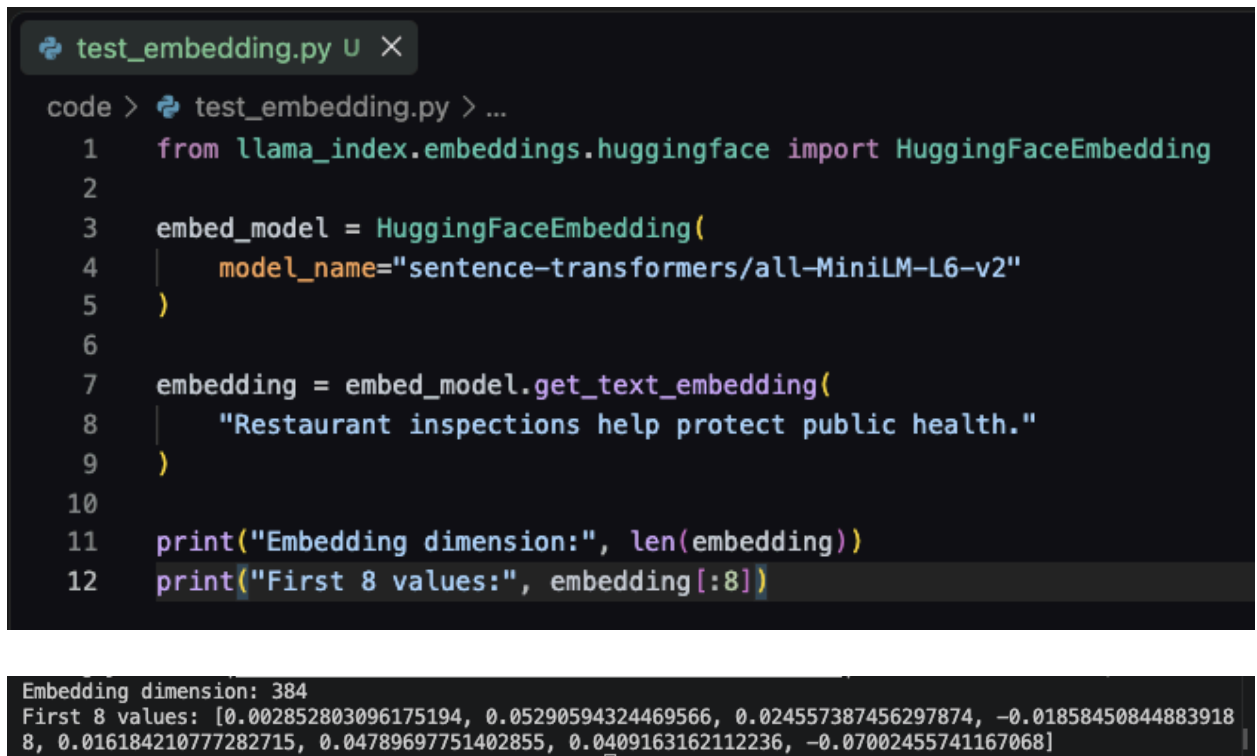
79     )
80
81
82 @router.post("/login")
83 def login(request: Request, username: str = Form(...), password: str = Form(...)):
84     """
85     Handles login form submission.
86
87     - Reads username and password from the form
88     - Validates credentials
89     - Stores user info in session if valid
90     """
91     if username == VALID_USERNAME and password == VALID_PASSWORD:
92         request.session["user"] = username
93         request.session["last_activity"] = time.time()
94
95         return RedirectResponse(
96             url="/dashboard",
97             status_code=HTTP_302_FOUND
98         )
99
100     return RedirectResponse(
101         url="/login?error=1",
102         status_code=HTTP_302_FOUND
103     )
104
105
106 @router.get("/dashboard")
107 def dashboard(request: Request):
108     """
109     Protected route.
110
111     - Only accessible if user is logged in
112     - Redirects to login page if session is missing
113     """
114     user = get_active_user(request)
115
116     # If user is not logged in, block access
117     if not user:
118         return RedirectResponse(
119             url="/login?expired=1",
120             status_code=HTTP_302_FOUND
121         )
122
123     return templates.TemplateResponse(
124         request,
125         "dashboard.html",
126         {
127             "user": user
128         }
129     )
130
131
132 @router.get("/logout")
133 def logout(request: Request):
134     """
135     Logs the user out.
136
137     - Clears all session data
138     - Redirects back to home page
139     """
140     request.session.clear()
141
142     return RedirectResponse(
143         url="/",
144         status_code=HTTP_302_FOUND
145     )

```

Part 2: Compare Three LlamaIndex Chunking Techniques (Retrieval-Only RAG)

A. Environment & Setup

I installed the required libraries: llama-index, llama-index-embeddings-huggingface, sentence-transformers, faiss-cpu, numpy, and pandas. For the embedding model, I used the public Hugging Face model sentence-transformers/all-MiniLM-L6-v2. I verified that the model loaded successfully and generated a 384-dimensional embedding for a sample restaurant-inspection sentence.



```

test_embedding.py U X
code > test_embedding.py > ...
1  from llama_index.embeddings.huggingface import HuggingFaceEmbedding
2
3  embed_model = HuggingFaceEmbedding(
4      model_name="sentence-transformers/all-MiniLM-L6-v2"
5  )
6
7  embedding = embed_model.get_text_embedding(
8      "Restaurant inspections help protect public health."
9  )
10
11 print("Embedding dimension:", len(embedding))
12 print("First 8 values:", embedding[:8])

```

```

Embedding dimension: 384
First 8 values: [0.002852803096175194, 0.05290594324469566, 0.024557387456297874, -0.01858450844883918
8, 0.016184210777282715, 0.04789697751402855, 0.0409163162112236, -0.07002455741167068]

```

Dataset/Corpus: I used three public FDA documents related to restaurant inspections and food safety as the graded corpus. The local snapshots total approximately 9.97 MB, exceeding the 200 KB minimum. Source URLs and access dates are recorded in SOURCES.md, while filenames, file sizes, and SHA-256 hashes are recorded in CORPUS_MANIFEST.json.

B. One retrieval-only pipeline per technique**1. Chunking**

- **Token**

I used a token chunk size of 256 with an overlap of 50 tokens to preserve some context between adjacent chunks.

```
hw3_retrieval.py U X
code > hw3_retrieval.py > ...
10
71 token_splitter = TokenTextSplitter(
72     chunk_size=256,
73     chunk_overlap=50
74 )
75
76 token_nodes = token_splitter.get_nodes_from_documents(
77     documents
78 )
79
80 print(f"\nToken chunks: {len(token_nodes)}")
81
```

Technique	Chunks
Token	3220

- **Semantic**

Semantic chunking used the same sentence embedding model to identify semantically coherent boundaries, with a buffer size of 1 and a 95th-percentile breakpoint threshold.

hw3_retrieval.py U X

code > hw3_retrieval.py > ...

```
83 semantic_splitter = SemanticSplitterNodeParser(  
84     buffer_size=1,  
85     breakpoint_percentile_threshold=95,  
86     embed_model=embed_model  
87 )  
88  
89 semantic_nodes = semantic_splitter.get_nodes_from_documents(  
90     documents  
91 )  
92  
93 print(f"Semantic chunks: {len(semantic_nodes)}")  
94
```

Technique	Chunks
Token	3220
Semantic	2253

- **Sentence-window**

Sentence-window chunking split the documents into individual sentences and attached a window of three neighboring sentences as metadata. This keeps each indexed chunk small while preserving surrounding context that can be used to better understand the retrieved sentence.

```

hw3_retrieval.py U X
code > hw3_retrieval.py > ...
96     sentence_splitter = SentenceWindowNodeParser.from_defaults(
97         window_size=3,
98         window_metadata_key="window",
99         original_text_metadata_key="original_text"
100     )
101
102     sentence_nodes = sentence_splitter.get_nodes_from_documents(
103         documents
104     )
105
106     print(
107         f"Sentence-window chunks: "
108         f"{len(sentence_nodes)}"
109     )

```

Technique	Chunks
Token	3220
Semantic	2253
Sentence Window	17016

```

48 Sentence-window example:
49 Indexed sentence:
50 Managing Food Safety:
51 A Regulator's Manual
52 For Applying HACCP
53 Principles to Risk-
54 based Retail and Food
55 Service Inspections
56 and Evaluating
57 Voluntary Food Safety
58 Management Systems
59
60 Additional copies are available from:
61
62 Office of Food Safety
63 | Retail Food and Cooperative Programs
64 | | Coordination Staff/Retail Food Protection Team (HFS-320)
65 | | | Center for Food Safety and Applied Nutrition
66 | | | | Food and Drug Administration
67 | | | | | 5100 Paint Branch Parkway
68 | | | | | College Park, MD 20740
69 | | | | | (Tel) 240-402-2349
70
71
72
73 U.S.
74
75 Window metadata:
76 Managing Food Safety:
77 A Regulator's Manual
78 For Applying HACCP
79 Principles to Risk-
80 based Retail and Food
81 Service Inspections
82 and Evaluating
83 Voluntary Food Safety
84 Management Systems
85

```

2. Indexing (in memory)

For each chunking technique, I created a separate in-memory SimpleVectorStore and used it with StorageContext to build a VectorStoreIndex. The same embedding model was used for all three indexes so the retrieval comparison remained consistent and fully local.

 hw3_retrieval.py U X

```
code >  hw3_retrieval.py >  cosine_similarity
170 token_vector_store = SimpleVectorStore()
171
172 token_storage = StorageContext.from_defaults(
173     vector_store=token_vector_store
174 )
175
176 token_index = VectorStoreIndex(
177     token_nodes,
178     storage_context=token_storage,
179     embed_model=embed_model
180 )
181
182
183 semantic_vector_store = SimpleVectorStore()
184
185 semantic_storage = StorageContext.from_defaults(
186     vector_store=semantic_vector_store
187 )
188
189 semantic_index = VectorStoreIndex(
190     semantic_nodes,
191     storage_context=semantic_storage,
192     embed_model=embed_model
193 )
194
195
196 sentence_vector_store = SimpleVectorStore()
197
198 sentence_storage = StorageContext.from_defaults(
199     vector_store=sentence_vector_store
200 )
201
202 sentence_index = VectorStoreIndex(
203     sentence_nodes,
204     storage_context=sentence_storage,
205     embed_model=embed_model
206 )
```

3. Retrieval-only function

I implemented a retrieval helper that takes a query and top-k value, computes the query embedding, retrieves the top-k chunks, explicitly embeds each returned chunk, computes cosine similarity, records the store score and chunk length, prints vector shapes, and saves the results as JSON.

```

hw3_retrieval.py U X
code > hw3_retrieval.py > run_retrieval
244 def run_retrieval(
274     query_embedding = np.array(
275         embed_model.get_query_embedding(
276             query
277         ),
278         dtype=float
279     )
280
281     print(
282         f"Query embedding dimension: "
283         f"{len(query_embedding)}"
284     )
285
286     print(
287         "First 8 query embedding values:",
288         query_embedding[:8].tolist()
289     )
290
291     retriever = index.as_retriever(
292         similarity_top_k=k
293     )
294
295     start_time = time.perf_counter()
296
297     results = retriever.retrieve(
298         query
299     )
300
301     latency_ms = (
302         time.perf_counter()
303         - start_time
304     ) * 1000
305
306     rows = []
307     doc_vectors = []
308
309     for rank, result in enumerate(
310         results,
311         start=1
312     ):
313         chunk_text = result.node.get_content(
314             metadata_mode=MetadataMode.NONE
315         )
316
317         doc_embedding = np.array(
318             embed_model.get_text_embedding(
319                 chunk_text
320             ),
321             dtype=float
322         )
323
324         doc_vectors.append(
325             doc_embedding
326         )
327
328         cosine_sim = cosine_similarity(
329             query_embedding,
330             doc_embedding
331         )
332
333         preview = " ".join(
334             chunk_text.split()
335            )[:160]
336

```

```
hw3_retrieval.py U X
code > hw3_retrieval.py > run_retrieval
244 def run_retrieval(
363
364     row = {
365         "rank": rank,
366         "store_score": (
367             float(result.score)
368             if result.score is not None
369             else None
370         ),
371         "cosine_sim": cosine_sim,
372         "chunk_len": len(
373             chunk_text
374         ),
375         "preview": preview,
376         "source_file": source_file,
377         "window_context": (
378             window_context
379         )
380     }
```

```
hw3_retrieval.py U X
code > hw3_retrieval.py > run_retrieval
244 def run_retrieval(
398
399     print(
400         f"Query vector shape: "
401         f"{query_embedding.shape}"
402     )
403
404     print(
405         f"Stacked document vector shape: "
406         f"{doc_matrix.shape}"
407     )
408
```

```

hw3_retrieval.py U X
code > hw3_retrieval.py > ...
244 def run_retrieval(
480     with open(
481         output_file,
482         "w",
483         encoding="utf-8"
484     ) as f:
485         json.dump(
486             output,
487             f,
488             indent=2,
489             ensure_ascii=False
490         )
491

```

This example shows the sentence-window pipeline retrieving the top five chunks and reporting the store score, independently computed cosine similarity, chunk length, and text preview for each result.

```

sentence_window_q1.json U X
reports > hw03 > raw > sentence_window_q1.json > ...
1 {
2   "technique": "sentence_window",
3   "question_id": "q1",
4   "query": "What are the three major activities in the verification process for a voluntary food safety management system?",
5   "expected_answer": "The three major activities are document review, record review, and on-site verification.\n",
6   "expected_source": "A-Regulator's-Manual-For-Applying-MACCP-Principles-to-Risk-based-Retail-and-Food-Service-Inspections-and-Evaluating-Voluntary-Food-Safety-Management-Systems.pdf",
7   "k": 5,
8   "query_embedding_dimension": 384,
9   "query_embedding_first_8": [
10    0.022736009304089546,
11    0.016523387283086777,
12    -0.03188064321875572,
13    -0.04794027656316757,
14    0.05016084015369415,
15    0.006033062003552914,
16    0.02393440529704094,
17    -0.0268405731767416
18  ],
19   "query_vector_shape": [
20    384
21  ],
22   "doc_vector_shape": [
23    5,
24    384
25  ],
26   "latency_ms": 132.35479200375266,
27   "results": [
28     {
29       "rank": 1,
30       "store_score": 0.8374180885844049,
31       "cosine_sim": 0.8374180892242347,
32       "chunk_len": 179,
33       "preview": "THE VERIFICATION PROCESS The verification of food safety management systems involves three major activities: • Document Review • Record Review • On-site Verific",
34       "source_file": "A-Regulator's-Manual-For-Applying-MACCP-Principles-to-Risk-based-Retail-and-Food-Service-Inspections-and-Evaluating-Voluntary-Food-Safety-Management-Systems.pdf",
35       "window_context": "Keep in mind that that there are many different types of food safety management \n systems. Some may control foodborne illness risk factors using only some of the \n principles of MACCP."
36     }
37   ]
38 }

```

Representative retrieval output (Sentence-window, Q1): The output identifies the technique, shows the query embedding dimension and vector shapes, and lists the top five retrieved chunks with rank, store score, cosine similarity, chunk length, and preview.

```
=====
Technique: sentence_window
Question ID: q1
Query: What are the three major activities in the verification process for a voluntary food safety management system?
=====
Query embedding dimension: 384
First 8 query embedding values: [0.0227736089304089546, 0.016523387283086777, -0.03188064321075572, -0.04794027656316757, 0.05016084015369415, 0.006833062003552914, 0.02393440529704094, -0.0268485731767416]
Query vector shape: (384,)
Stacked document vector shape: (5, 384)
Retrieval latency: 132.355 ms

Rank  Store Score  Cosine Sim  Chunk Len  Preview
-----
1      0.837418      0.837418      179      THE VERIFICATION PROCESS The verification of food safety management systems involves three major activities: • Document Review • Record Review • On-site Verific
2      0.789988      0.789988      223      Reviewing a voluntary food safety management system to determine whether the corrective actions and the monitoring, verification, and record keeping procedures
3      0.767192      0.767192      265      Validation and verification of voluntarily implemented food safety management systems are services you can offer your industry partners to provide them with fee
4      0.745458      0.745458      83       Elements of an effective food safety management system may include the following:
5      0.742994      0.742994      1233     Generally, a written, voluntary food safety management system developed using the FDA document entitled, Managing Food Safety: A Manual for the Voluntary Use of

Saved raw output: reports/hw03/raw/sentence_window_q1.json
=====
```

Retrieval Quality Comparison

1. Retrieval Quality:

Sentence-window chunking produced the strongest retrieval similarity scores, with the highest mean top-1 cosine (0.8224) and mean@5 cosine (0.7695). All three techniques achieved Recall@5 of 1.00, meaning the expected source appeared somewhere in the top five results for every question. However, sentence-window chunking required substantially more chunks and had much higher retrieval latency. Semantic chunking was the fastest approach, while token chunking produced intermediate latency and index size.

Technique	Chunks	Avg chunk length	Top-1 cosine	Mean@5 cosine	Recall@5	Mean retrieval latency (ms)
Token	3220	852.20	0.7529	0.7201	1.00	27.53
Semantic	2253	1016.20	0.7469	0.7141	1.00	20.15
Sentence Window	17016	134.55	0.8224	0.7695	1.00	128.16

*Average chunk length is measured in characters, and all metrics were recomputed from the machine-readable retrieval outputs in reports/hw03/raw/.

Confidently Scored Incorrect Retrieval:

For Q3, “What is the purpose of the FDA Food Code?”, the sentence-window technique returned the passage “It is recommended that it be used in conjunction with the FDA Food Code.” as rank 1 with a cosine similarity of 0.8411. Although the score was high, the passage does not answer the question. The embedding model likely considered it highly similar because the sentence directly contains “FDA Food Code” and is semantically related to the query topic, demonstrating that high cosine similarity does not necessarily mean that the passage contains the answer.

2. Observations

Sentence-window chunking performed the best overall, because its smaller sentence-level chunks were often more focused on the exact information needed by the query. This likely helped the embedding model match the question to highly relevant passages without unrelated surrounding text. The attached sentence window also preserved nearby context, so the retrieved sentence could remain concise while still retaining useful neighboring information. In contrast, the token and semantic chunking produced much longer average chunks, which may have mixed relevant and less relevant content and slightly reduced cosine similarity.

The tradeoff however was efficiency. Sentence-window chunking created 17,016 chunks and had the highest mean retrieval latency at 128.16 ms, while semantic chunking was much faster at 20.15 ms with only 2,253 chunks. Although sentence-window had the strongest average retrieval scores across the five questions, the Q3 result also showed that a short sentence can receive a high similarity score simply because it mentions the same topic without actually answering the question. This shows that focused chunks can improve relevance, but cosine similarity alone does not guarantee answer correctness.

3. Conclusion

For this corpus, sentence-window chunking performed best overall, with the highest mean top-1 cosine (0.8224) and mean@5 cosine (0.7695), while maintaining Recall@5 of 1.00. Although it had higher latency and produced more chunks, its retrieval-quality measurements were strongest overall.

AI USE:

1. What did you use an AI assistant for, and what did you do yourself?

I used an AI assistant as a support tool throughout the assignment. It helped me interpret the homework requirements, debug and test parts of the retrieval pipeline, identify and fix implementation issues, and improve syntax, formatting, grammar, wording, and organization in my code and report. It also helped draft and format supporting Markdown and text files based on my implementation, outputs, and results. I remained responsible for the overall implementation and decisions: I set up the repository, selected and downloaded the corpus, ran and tested the code, verified the outputs and retrieval results, created the screenshots, and assembled the final report.

2. Give one AI-produced output that was wrong or unsuitable, or one result you independently verified.

One initial approach for loading the PDF corpus was unsuitable because it resulted in PDF binary and internal file syntax being treated as document content instead of clean extracted text.

3. How did you detect the problem or verify the result?

I detected the issue because the output contained strings such as %PDF-1.6 and other unreadable PDF-internal content instead of normal English text from the FDA documents. The initial ingestion also produced only a few document objects, which was inconsistent with a corpus containing more than one thousand PDF pages.

4. What did you change, and why does it work now?

I changed the ingestion process to use pypdf.PdfReader and explicitly extract text from each page before creating LlamaIndex Document objects. After this change, the pipeline loaded 1,023 document/page objects and produced readable FDA text for chunking and retrieval. This works better because the retrieval system now embeds actual document text instead of PDF file structure or binary content.